

Seckill: A High-Concurrency Flash Sale System

s2269587

April 2026

1 Motivation and Target Audience

Flash sales — where limited inventory must be sold to thousands of buyers within seconds — are among the most demanding scenarios in e-commerce. Platforms such as JD.com and Taobao handle massive traffic during events like 11.11, where naive implementations can lead to overselling, duplicate orders, and cascading failures.

A synchronous design, where a single request performs stock validation, inventory update, order creation, payment, and downstream processing, quickly becomes a bottleneck. It increases latency and concentrates write contention on the database, which is difficult to sustain under high concurrency.

This project builds a flash sale system in Go as an engineering prototype for e-commerce platforms and retailers running limited-stock promotions. It focuses on how correctness and throughput can be maintained under contention through careful stock control, asynchronous processing, and order lifecycle management.

2 Core Idea and Innovation

The main idea of this project is not simply to simulate an online sale, but to build a focused prototype around the hardest parts of a flash sale system. In this setting, the key challenge is preserving correctness when thousands of users compete for a very small amount of stock at the same time.

The system addresses several failure-prone points together: preventing overselling, blocking duplicate orders, handling unpaid orders automatically, and allowing downstream services to react to order events independently. This is valuable for e-commerce platforms and retailers running limited-stock promotions, where both throughput and correctness are essential.

A central design feature is the separation of responsibilities across different components. Redis is used for hot stock control, Kafka for event fan-out, and RabbitMQ for delayed timeout cancellation. Instead of forcing one broker or database to handle every part of the workflow, the system assigns each technology to the task it supports best. This makes the project more than a simple demo: it is a compact engineering model of how high-concurrency sales systems can be designed in a more realistic and maintainable way.

3 Architecture and Design Decisions

The system is structured as a set of loosely coupled services, with design decisions centred on reducing contention in the request path and separating time-sensitive operations from background processing.

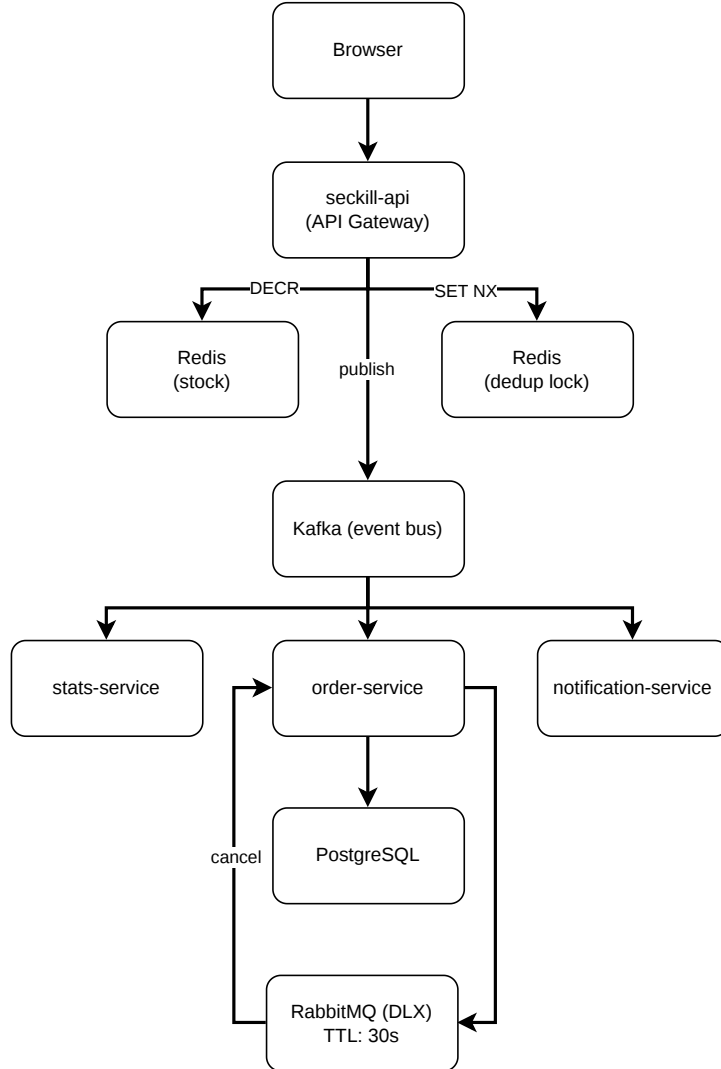


Figure 1: System architecture illustrating the separation of concerns between request handling, event propagation, and delayed task management in a high-concurrency flash sale system

- **Stock management (Redis vs PostgreSQL):** Real-time stock is maintained in Redis, while PostgreSQL stores the authoritative order state. This avoids high contention on database rows during peak traffic, as in-memory atomic operations can handle frequent updates more efficiently. Relying solely on database updates would introduce locking and significantly reduce throughput under concurrent access.
- **Event propagation (Kafka):** Kafka is used as a high-throughput event bus. When an order is created, an event is published and consumed by multiple services independently. This decouples order processing from downstream concerns such as statistics and notifications, and allows new consumers to be added without modifying existing producers.
- **Timeout handling (RabbitMQ):** Order timeout cancellation is handled using Rab-

bitMQ’s delayed message mechanism. Orders that are not paid within a fixed window are automatically routed to a cancellation queue. This avoids the need for polling or periodic schedulers and ensures that timeout logic is handled asynchronously and reliably.

- **Consistency under concurrency:** To handle race conditions between payment and cancellation, the system uses conditional updates at the database level. An order is only cancelled if it remains in a pending state, ensuring that late payments are not incorrectly overridden by timeout events.

These decisions reflect a separation of concerns across system components, avoiding tightly coupled request flows and allowing the system to scale and evolve more predictably under load.

4 Implementation and Completeness

The system implements a complete end-to-end flash sale workflow. Users submit purchase requests through the API, with stock validation and order creation handled under high concurrency. Duplicate submissions are limited through a Redis-based short-lived deduplication key, ensuring that repeated requests from the same user do not create multiple concurrent orders.

Successful orders are propagated through Kafka, allowing independent services to update statistics and record events without affecting the request path. Unpaid orders are automatically cancelled after a fixed timeout(30s) using RabbitMQ, completing the order lifecycle without requiring manual intervention or scheduled jobs. This ensures that stock can be released in a timely manner and reused by other users.

The system is fully deployable using Docker Compose, with all services running in a reproducible environment. A lightweight dashboard provides real-time visibility into request throughput and order outcomes, including successful, rejected, and cancelled orders under load. In a representative load test with 10,000 concurrent users competing for 100 items, the system processed requests at approximately 5,000 QPS and produced exactly 100 successful orders, with all remaining requests correctly rejected. This behaviour, also reflected in the real-time dashboard, demonstrates that the system preserves correctness by preventing overselling while maintaining stable throughput under high contention.

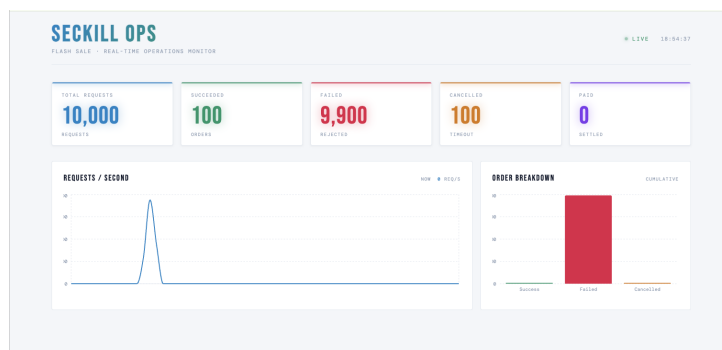


Figure 2: Real-time dashboard showing request throughput and order outcomes during a high-concurrency load test

5 AI Usage

Claude Code was used throughout the project as a supporting development aid. In particular, they were helpful in identifying subtle issues such as Redis command behaviour under conditional set operations, understanding Kafka configuration problems in containerised environments, and reasoning about concurrency edge cases such as the race between payment and timeout cancellation.

In each case, the generated suggestions were reviewed and validated before being incorporated. The use of AI therefore complemented the development process by accelerating debugging and exploration, while the final design decisions and system structure were determined through independent reasoning and verification.